

IDV-ASAS WebUI 版实现原理文档

1. 项目概述

IDV-ASAS WebUI 版是《第五人格》区域选择辅助系统的 Web 界面版本。与桌面 GUI 版（`app/`）使用 `eframe/egui` 渲染桌面窗口不同，WebUI 版使用 **Axum** 构建 Rust 本地 Web 服务，以 **REST API + SSE** 暴露后端能力，以 **HTML/CSS/JavaScript** 实现浏览器端界面。

核心设计原则：**后端逻辑模块完全复用桌面版的 Rust 源码，不做任何修改**。7 个后端模块（`api.rs`、`config.rs`、`checker.rs`、`positions.rs`、`prompt.rs`、`history.rs`、`parser.rs`）被原样复制到 `webui-ver/src/` 中。新增的仅有：

- `main.rs` — Axum Web 服务（路由、状态管理、SSE 流式分析）
- `static/index.html` — 前端主页面
- `static/style.css` — 样式文件
- `static/app.js` — 前端逻辑

桌面版中依赖 `egui` 的 `app.rs`（状态机 + `eframe::App`）、`ui_impl.rs`（全部 UI 渲染）、`images.rs`（`TextureHandle` 缓存）被完全移除，由上述 Web 技术栈替代。

2. 技术选型

层面	技术	说明
语言	Rust 1.75+	后端必须使用 Rust (resources/AGENTS.md 要求)
Web 框架	axum 0.8	异步 Web 框架, 提供路由、JSON 提取、SSE 响应
异步运行时	tokio 1 (full features)	Axum 的异步运行时, 提供 <code>Mutex</code> 、 <code>time::sleep</code> 、 <code>TcpListener</code>
静态文件	tower-http 0.6 (ServeDir)	提供 <code>static/</code> 和 <code>../icons/</code> 目录的文件服务
HTTP 客户端	ureq 3.4	与桌面版相同, OpenAI 兼容的流式 API 调用
序列化	serde + serde_json	与桌面版相同, 配置与历史记录的 JSON 读写
时间	chrono 0.4	与桌面版相同, 历史记录的时间戳命名
流式支持	async-stream 0.3	<code>stream!</code> 宏, 用于生成 SSE 事件流
前端语言	HTML + CSS + JavaScript	纯原生, 无框架依赖
前端字体	Google Fonts (Noto Sans SC)	中文字体支持; 离线时回退到系统字体
图片渲染	HTML5 Canvas 2D	替代桌面版的 egui ColorImage + TextureHandle
事件推送	SSE (Server-Sent Events)	替代桌面版的 <code>mpsc::Receiver</code> 轮询 + <code>ctx.request_repaint()</code>

关键差异说明

桌面版通过 `eframe::App::logic()` 每帧轮询 `mpsc::Receiver`, 并通过 `ctx.request_repaint()` 驱动重绘。WebUI 版将此机制改为 **SSE 长连接**: 前端通过 `EventSource` 建立连接, 服务端在 async stream 中轮询同一个 `mpsc::Receiver`, 通过 SSE 事件向前端推送结果。

3. 目录结构

webui-ver/	
├ Cargo.toml	# 依赖声明与编译配置
├ README_WEBUI.md	# 本目录的 README
├ DESIGN_WEBUI.md	# 本文件
├ config.json	# 运行时生成, API 配置 (明文存储)
├ history/	# 运行时生成, 会话历史 JSON 文件
├ static/	# 前端静态文件 (由 tower-http ServeDir 提供服务)
├── index.html	# 主页面 (全部 12 个步骤界面)
├── style.css	# 样式 (深黑蓝 + 黄色主题, 与桌面版配色一致)
└ app.js	# 前端逻辑 (步骤流程 + Canvas 图片合成 + SSE 消费)
└ src/	
├── main.rs	# web 服务入口: 路由定义、AppState 状态管理、SSE 分析处理器
└ api.rs	# ← 与桌面版完全相同

└─ config.rs	# ← 与桌面版完全相同
└─ checker.rs	# ← 与桌面版完全相同
└─ positions.rs	# ← 与桌面版完全相同
└─ prompt.rs	# ← 与桌面版完全相同
└─ history.rs	# ← 与桌面版完全相同
└─ parser.rs	# ← 与桌面版完全相同

后端模块复用验证

7 个后端模块通过 `diff` 验证与 `app/src/` 中的对应文件逐字节一致：

```
api.rs:      identical
config.rs:   identical
checker.rs:  identical
positions.rs: identical
prompt.rs:   identical
history.rs:  identical
parser.rs:   identical
```

main.rs 的角色

`main.rs` 承担了桌面版中 `app.rs` 的状态管理职责和 `ui_impl.rs` 的流程编排职责，但不包含任何渲染逻辑。具体包括：

1. **AppState 结构体**：持有会话状态（配置、选图、选角、需求、消息历史、分析结果等），替代桌面版 `IdvApp` 结构体
2. **Axum 路由**：14 个 REST API 端点 + 2 个静态文件目录，替代桌面版的 `match self.step` 状态分发
3. **SSE 分析处理器**：在 `async stream` 中轮询 `mpsc::Receiver`，处理重试逻辑，替代桌面版的 `poll_api()` + `ctx.request_repaint()`
4. **序列化辅助函数**：将 `Plan`、`Usage` 等 Rust 结构体手动序列化为 JSON，替代桌面版的 egui `RichText` / `Image` 渲染

4. 资源路径映射

程序运行时以 `CARGO_MANIFEST_DIR`（即 `webui-ver/`）为基准，向上一级定位项目根目录，再访问各资源：

资源	运行时路径	访问方式	用途
角色数据	<code>../resources/characters/*.md</code>	Rust 文件 I/O	六维属性、天赋推荐、联动关系
地图数据	<code>../resources/maps/*.md</code>	Rust 文件 I/O	选点俗称、牵制难易、选点推荐模式
通用资料	<code>../resources/README.md</code> 等	Rust 文件 I/O	求生者属性构成、天赋选择一般规律
字典	<code>../DICTIONARY.md</code>	Rust 文件 I/O	角色中英文名映射、地图中英文名映射、天赋中英文名映射
概念图	<code>../icons/maps/*.png</code>	HTTP <code>/icons/maps/*.png</code>	地图选择界面的 3×3 网格图片
区域选点图	<code>../icons/area-selection/*.png</code>	HTTP <code>/icons/area-selection/*.png</code>	结果展示界面的底图
角色头像	<code>../icons/characters/*.png</code>	HTTP <code>/icons/characters/*.png</code>	角色选择与结果展示的头像 (原始 120×120)
天赋图标	<code>../icons/personas/*.png</code>	HTTP <code>/icons/personas/*.png</code>	结果展示的天赋图标
选点坐标	<code>../icons/positions/*.md</code>	Rust 文件 I/O → JSON API	各地图选点中心坐标与图标尺寸
配置文件	<code>webui-ver/config.json</code>	Rust 文件 I/O	API 配置 (明文存储)
历史记录	<code>webui-ver/history/*.json</code>	Rust 文件 I/O	会话历史 JSON 文件
前端文件	<code>webui-ver/static/*</code>	HTTP <code>/static/*</code>	index.html、style.css、app.js

路径访问的双重性

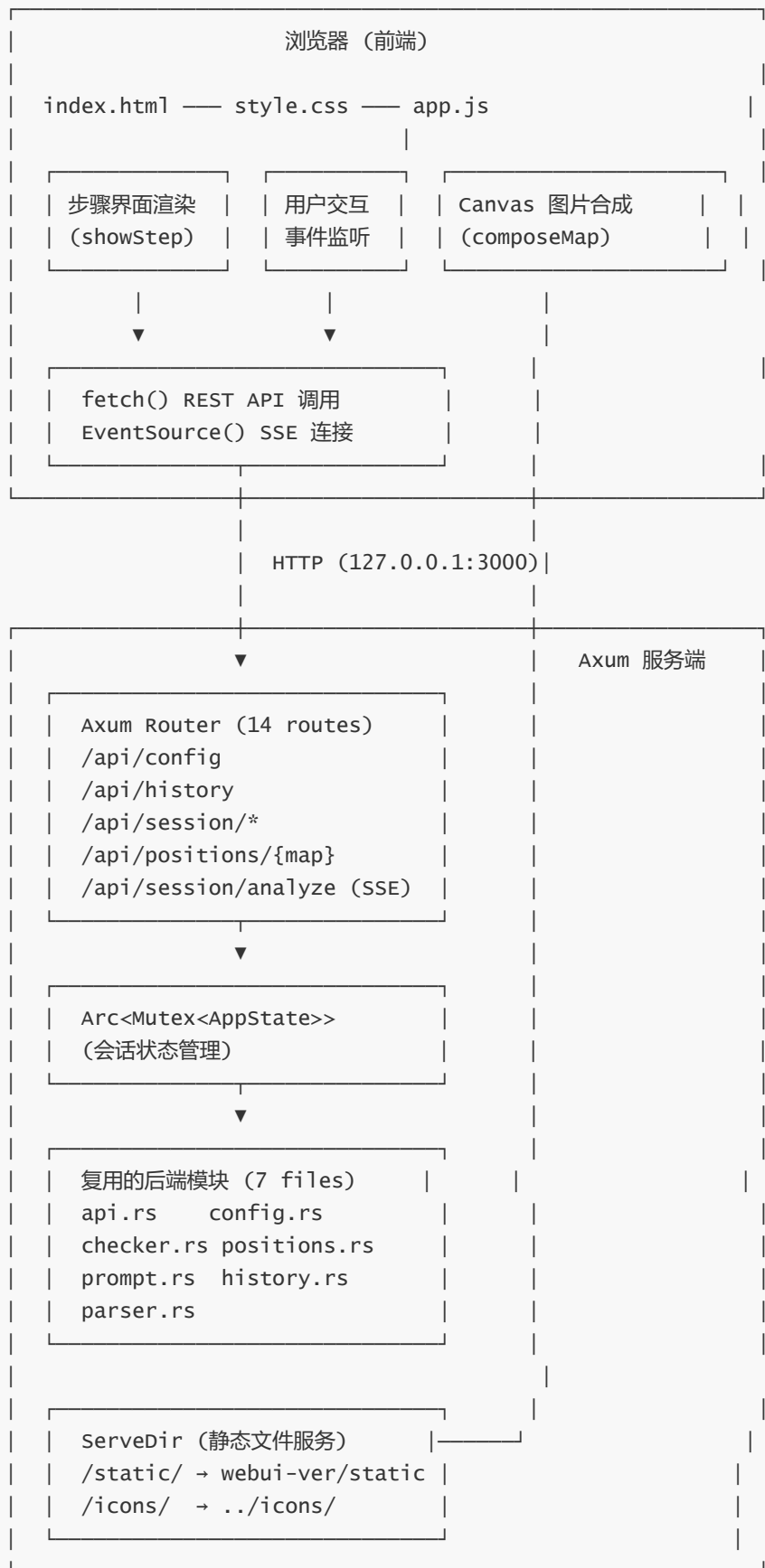
与桌面版不同，WebUI 版的资源存在两种访问路径：

1. **Rust 后端直接文件 I/O**：角色数据、地图数据、通用资料、字典、选点坐标等文本资源，由后端读取并注入模型 prompt 或序列化为 JSON API 响应。这些资源不需要通过 HTTP 暴露。
2. **HTTP 静态文件服务**：图片资源（概念图、选点底图、角色头像、天赋图标）和前端文件，通过 `tower_http::ServeDir` 对外暴露为 HTTP 路径，供浏览器直接访问。

```
// main.rs 中的路由配置
.nest_service("/static", ServeDir::new(app_root.join("static")))
.nest_service("/icons", ServeDir::new(project_root.join("icons")))
```

5. 架构设计

5.1 整体架构



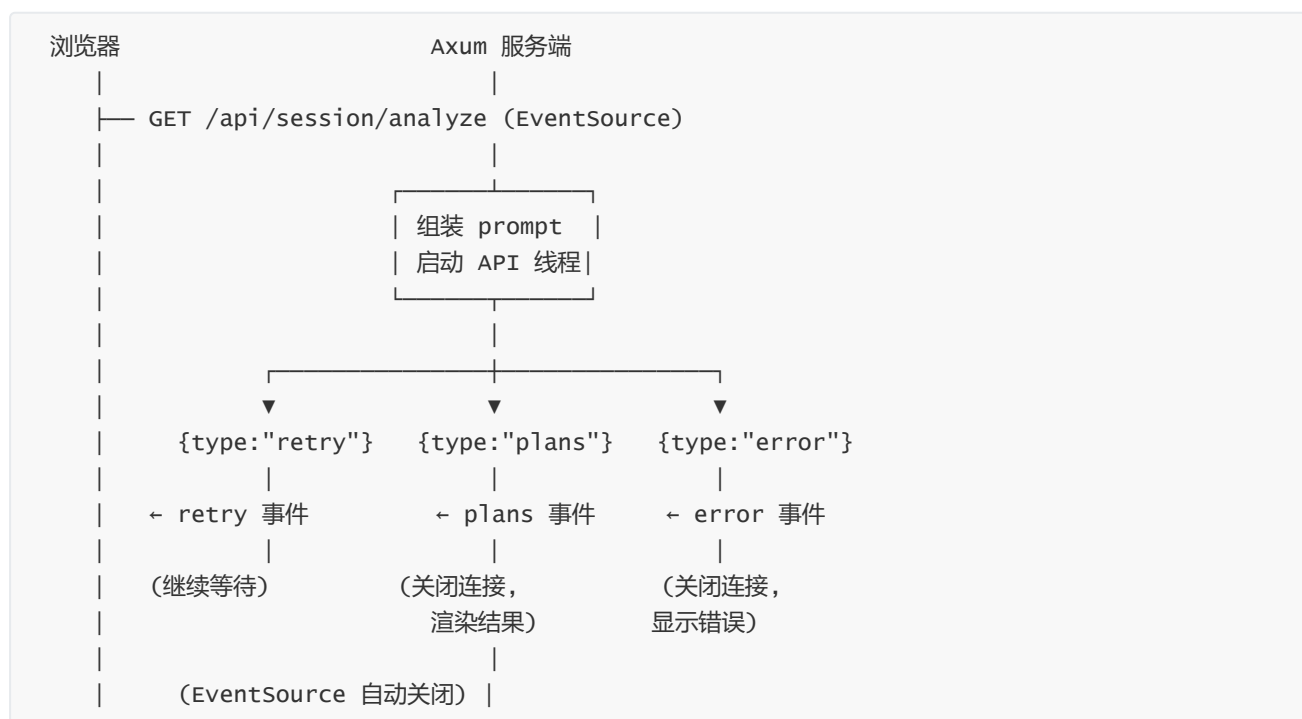
5.2 请求-响应模型

桌面版中，UI 渲染和业务逻辑在同一线程通过 `egui::App::logic()` 每帧轮询。WebUI 版将其拆分为经典的客户端-服务器模型：

普通操作（配置、历史、选图选角等）：REST API 请求-响应



分析操作：SSE 长连接流式推送



5.3 状态管理

WebUI 版使用 `Arc<Mutex<AppState>>` 作为全局共享状态，通过 Axum 的 `State` 提取器注入每个处理器：

```
struct AppState {
    // 资源
    project_root: PathBuf,
    db: checker::ResourceDb,           // 启动时加载一次
    system_prompt: String,           // 启动时组装一次

    // 配置
    config: config::ApiConfig,
    config_path: PathBuf,
```

```

// 会话
session: history::Session,
session_path: Option<PathBuf>,
history_dir: PathBuf,

// 当局选择
selected_map: String,
selected_chars: Vec<String>,
round_req: String,

// 模型调用
messages: Vec<Value>,          // 发送给模型的消息列表
retry_count: usize,           // 当前重试次数
raw_model_output: String,      // 模型原始输出
api_interrupt: Option<Arc<AtomicBool>>, // 打断标记

// 解析结果
plans: Vec<parser::Plan>,
current_plan_idx: usize,

// 统计
round_usage: Option<api::Usage>,
round_cost: f64,
round_estimated: bool,
}

```

与桌面版 `IdvApp` 结构体相比：

- 移除了所有 egui 相关字段（`egui_ctx`、`char_icons` / `map_icons` / `area_images` / `persona_icons` 的 `ImageCache`）
- 移除了前端 UI 状态字段（`cfg_show_form`、`cfg_use_default`、`save_name_input` 等，这些由前端 JS 管理）
- `step` 枚举被移除（状态流转由前端 JS 的 `showStep(name)` 控制）
- 核心业务字段（`session`、`config`、`selected_map`、`messages`、`plans` 等）保持一致

6. API 接口设计

6.1 路由总览

```

Router::new()
    .route("/", get(||
Redirect::permanent("/static/index.html")))
    .nest_service("/static", ServeDir::new(app_root.join("static")))
    .nest_service("/icons", ServeDir::new(project_root.join("icons")))
    .route("/api/config", get(get_config).post(set_config))
    .route("/api/history", get(list_history))
    .route("/api/history/load", post(load_history))
    .route("/api/history/save", post(save_history))
    .route("/api/session/global-req", post(set_global_req))
    .route("/api/resources", get(get_resources))

```

```
.route("/api/session/round",      post(set_round))
.route("/api/positions/{map}",    get(get_positions))
.route("/api/session/analyze",   get(analyze))           // SSE
.route("/api/session/interrupt", post(interrupt))
.route("/api/session/end-round", post(end_round))
.route("/api/session",          get(get_session))
.with_state(state)
```

6.2 端点详细说明

配置类

端点	方法	请求体	响应体	说明
/api/config	GET	—	{has_default, base_url, model, ...}	获取当前配置（检测是否有 config.json）
/api/config	POST	{base_url, api_key, model, temperature, max_tokens?, price_input_per_m, price_output_per_m, save_default}	{ok: true}	保存配置到内存和可选写文件

历史管理类

端点	方法	请求体	响应体	说明
/api/history	GET	—	[{name, model, rounds_count, prompt_tokens, completion_tokens, cost, global_req}, ...]	列出所有历史文件
/api/history/load	POST	{name: "xxx.json"}	{global_req, rounds_count} 或 {error}	加载历史到会话
/api/history/save	POST	{name: "xxx"}	{ok: true} 或 {error}	保存当前会话到文件

会话管理类

端点	方法	请求体	响应体	说明
<code>/api/session/global-req</code>	POST	<code>{text: "..."}</code>	<code>{ok: true}</code>	设置全局需求
<code>/api/session/round</code>	POST	<code>{map, characters, round_req}</code>	<code>{ok: true}</code>	设置当局地图、角色、需求
<code>/api/session</code>	GET	—	<code>{global_req, rounds_count, prompt_tokens, completion_tokens, cost, selected_map, selected_chars, round_req}</code>	获取当前会话状态
<code>/api/session/end-round</code>	POST	—	<code>{usage, cost, estimated, session_totals}</code>	结束当局，保存记录到会话

资源类

端点	方法	请求体	响应体	说明
<code>/api/resources</code>	GET	—	<code>{maps: [{stem, zh}], characters: [{stem, zh}], talent_map: {zh: stem}}</code>	获取地图、角色、天赋列表
<code>/api/positions/{map}</code>	GET	—	<code>{map_width, map_height, icon_size, points: [{num, x, y}]}</code>	获取地图选点坐标

分析类

端点	方法	请求体	响应体	说明
<code>/api/session/analyze</code>	GET	—	<code>text/event-stream</code> (SSE)	启动模型分析，返回 SSE 事件流
<code>/api/session/interrupt</code>	POST	—	<code>{ok: true}</code> 或 <code>{error}</code>	打断正在进行的分析

6.3 SSE 事件格式

`/api/session/analyze` 端点返回 `text/event-stream` MIME 类型的 SSE 流。每个事件的 `data` 字段为 JSON 字符串：

事件类型	data 格式	触发条件
<code>retry</code>	<code>{"type": "retry", "count": N}</code>	模型输出校验失败，自动重试第 N 次
<code>plans</code>	<code>{"type": "plans", "plans": [...], "usage": {...}, "cost": N, "estimated": bool}</code>	模型输出校验通过，返回有效方案
<code>error</code>	<code>{"type": "error", "message": "..."} </code>	分析被打断、API 错误、重试超限等

前端通过 `EventSource` 消费这些事件：

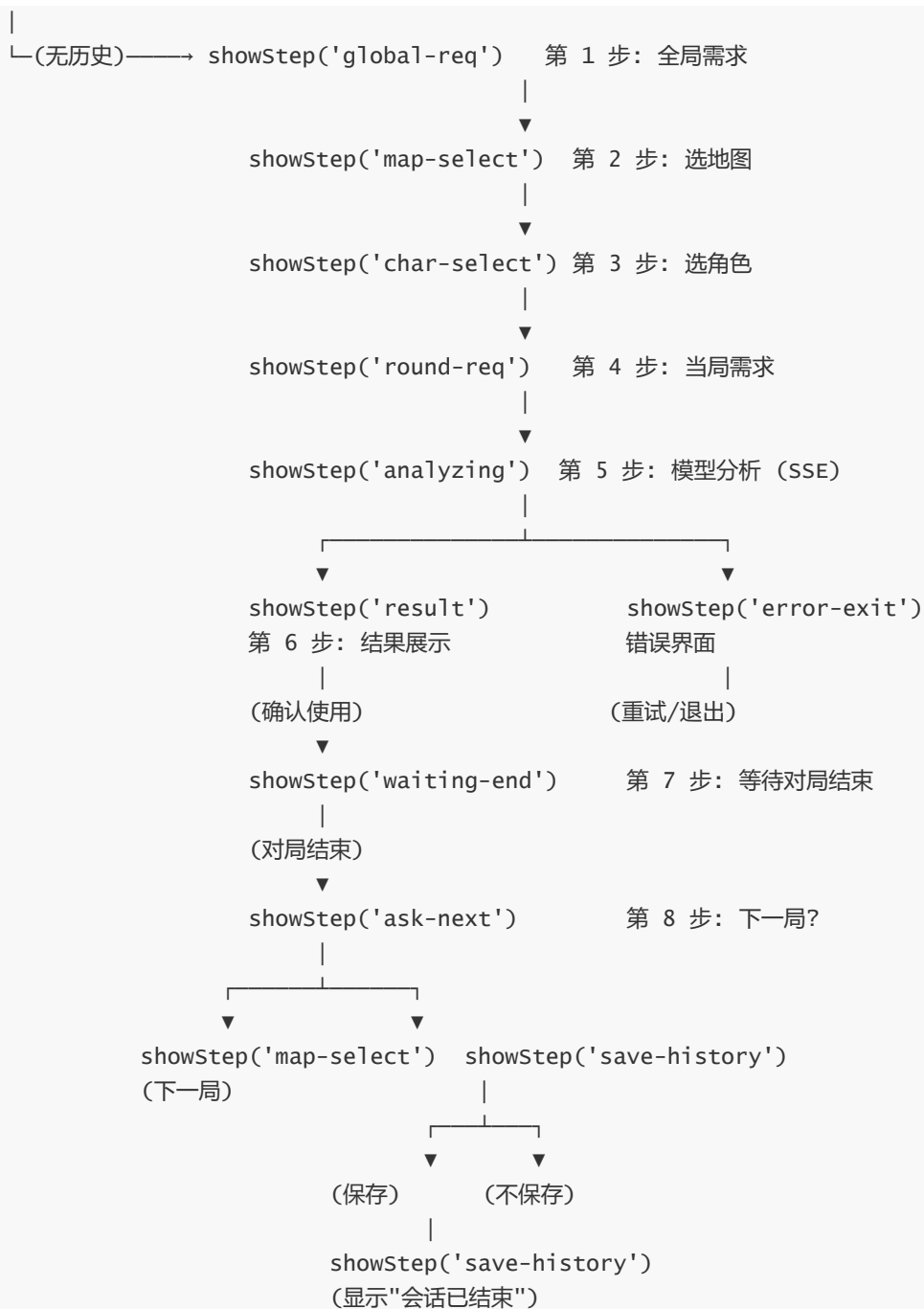
```
currentEvtSource = new EventSource('/api/session/analyze');
currentEvtSource.onmessage = (e) => {
  const data = JSON.parse(e.data);
  if (data.type === 'retry') { /* 显示重试提示 */ }
  else if (data.type === 'plans') { /* 渲染方案，关闭连接 */ }
  else if (data.type === 'error') { /* 显示错误，关闭连接 */ }
};
```

7. 前端状态机

7.1 前端步骤流转

与桌面版的 Rust `Step` 枚举不同，WebUI 版的状态流转完全由前端 JavaScript 的 `showStep(name)` 函数控制。每个步骤对应 HTML 中的一个 `<div class="step">`，通过显示/隐藏实现切换：

```
showStep('config')          第 0 步：API 配置
|
├─(沿用/确认)→ showStep('history')    历史导入
|
|                               |
├─(选择导入)→ showStep('map-select')
└─(跳过)→ showStep('global-req')
```



7.2 前端全局状态

前端使用全局变量管理会话状态，与后端 `AppState` 存在镜像关系：

```
let resources = null;           // /api/resources 缓存 (地图、角色、天赋列表)
let charZhToStem = {};         // 角色中文名 → 英文 stem 映射
let talentStemMap = {};        // 天赋中文名 → 英文 stem 映射
let selectedMap = '';          // 当前选中的地图 stem
let selectedChars = [];         // 当前选中的角色 stem 列表
let charPage = 0;              // 角色选择当前页码
let plans = [];                // 模型返回的方案列表
let currentPlanIdx = 0;         // 当前查看的方案索引
let roundUsage = null;         // 本局 Token 用量
let roundCost = 0;             // 本局费用
```

```
let roundEstimated = false;    // 是否为估算用量
let currentEvtSource = null;    // 当前 SSE 连接
let sessionRoundsCount = 0;    // 已完成的局数
let hasUnsavedSession = false; // 是否有未保存会话
```

7.3 前后端状态同步

前端状态与后端 `AppState` 通过 REST API 同步，但同步时机和方式有区别：

操作	前端状态变更	后端状态同步
配置 API	<code>window._lastConfig</code> 更新	<code>POST /api/config</code> 写入 <code>AppState.config</code>
设置全局需求	立即切换到地图选择界面	<code>POST /api/session/global-req</code> 写入 <code>session.global_req</code>
选地图/角色	<code>selectedMap</code> / <code>selectedChars</code> 更新，立即重渲染	<code>POST /api/session/round</code> 一次性写入（在第 4 步确认时）
模型分析	<code>currentEvtSource</code> 建立 SSE 连接	<code>GET /api/session/analyze</code> 启动分析线程
对局结束	<code>sessionRoundsCount</code> 更新	<code>POST /api/session/end-round</code> 保存 <code>RoundRecord</code>

注意：选地图和选角色在前端是分步的，但后端状态在确认当局需求时一次性提交（`POST /api/session/round`），减少 API 调用。

8. 核心模块原理（后端复用部分）

以下 7 个后端模块与桌面版（`app/src/`）逐字节一致，未做任何修改。此处仅做简要说明，详细原理参见桌面版 `app/DESIGN.md`。

8.1 提示词组装（prompt.rs）

系统提示词（`system_prompt`）：在服务启动时一次性组装，注入 `resources/AGENTS.md`、`DICTIONARY.md`、`resources/README.md`、`resources/characters/README.md`、`resources/maps/README.md` 及输出格式要求。存储于 `AppState.system_prompt`。

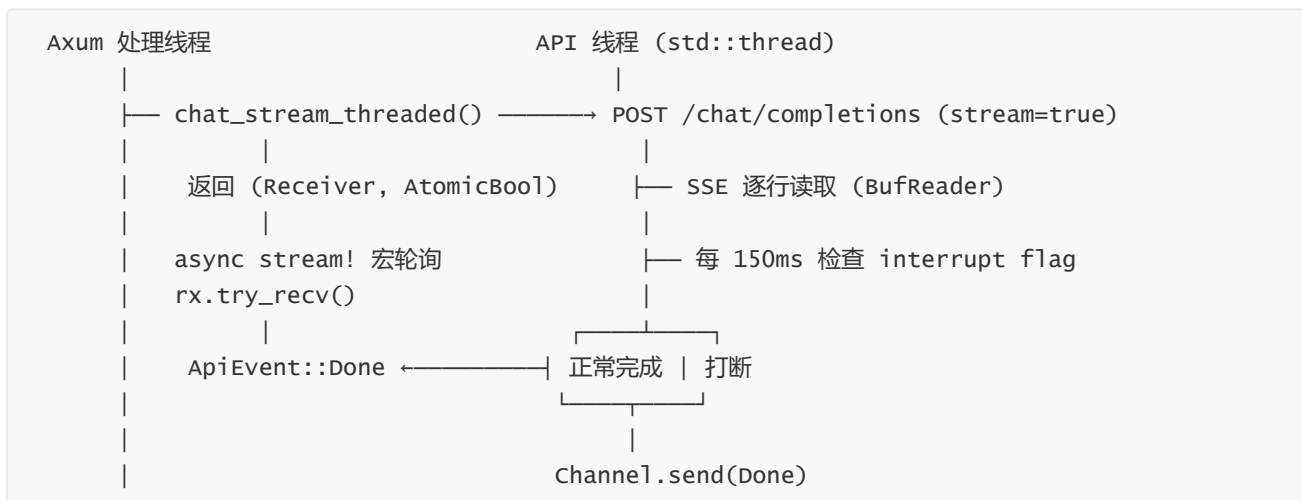
本局请求（`full_request`）：每局动态组装，包含紧凑信息 + 本局地图完整数据 + 四名角色完整数据。在 `analyze` 处理器中调用。

紧凑请求（`compact_request`）：用于历史局上下文，省略大段资料避免 Token 膨胀。在 `end_round` 处理器中调用。

与桌面版完全相同，包括系统提示词的文本内容、输出格式要求、覆盖说明等。

8.2 模型调用（api.rs）

采用**线程化流式调用**架构：



`chat_stream_threaded` 函数创建独立的 `std::thread` 进行 HTTP 请求（使用 `ureq`），通过 `mpsc::channel` 返回结果。打断通过 `Arc<AtomicBool>` 共享标记实现。

与桌面版完全相同，包括 120 秒超时检测、`stream_options` 兼容回退、Token 统计估算逻辑等。

8.3 输出解析与校验 (parser.rs)

模型返回文本经过三阶段处理：方案拆分 → 逐方案解析 → 合法性校验。校验失败时自动重试（最多 3 次），超限报告“您的模型与该项目不适配”。

与桌面版完全相同，包括 `split_plans`、`parse_single_plan`、`parse_selection_line`、`truncate_to_sentence` 等函数实现。

8.4 资源校验 (checker.rs)

启动时扫描 `resources/maps/*.md` 和 `resources/characters/*.md`，提取文件名和首行标题构建 `ResourceDb`。每个 `Entry` 包含中文名 (`zh`)、英文名 (`stem`) 和文件路径 (`path`)。

与桌面版完全相同。

8.5 选点坐标解析 (positions.rs)

解析 `icons/positions/*.md` 文件，提取地图原始尺寸、选点中心坐标和角色图标推荐尺寸。支持 LaTeX 格式 `\times` 与 Unicode `×` 两种写法。

与桌面版完全相同。

8.6 历史管理 (history.rs)

`Session` 结构包含版本号、创建时间、API 配置、全局需求、各局记录。`RoundRecord` 包含地图、角色、需求、紧凑请求、模型回答、Token 用量与费用。文件命名支持自定义名或时间戳。

与桌面版完全相同。

8.7 配置管理 (config.rs)

`ApiConfig` 包含 Base URL、API Key、模型名、温度、`max_tokens`、输入/输出单价。端点自动补全 `/chat/completions`。温度发送前四舍五入至 2 位小数。

与桌面版完全相同。

9. SSE 流式分析

9.1 设计动机

桌面版通过 `eframe::App::logic()` 每帧调用 `poll_api()` 来轮询 `mpsc::Receiver`，并通过 `ctx.request_repaint()` 驱动 UI 重绘。Web 版没有“每帧轮询”的机制，需要一种方式将 API 线程的结果推送到浏览器。

选择 **SSE (Server-Sent Events)** 而非 WebSocket 的原因：

- 单向通信（服务端 → 浏览器），不需要浏览器向服务端推送数据
- 浏览器原生 `EventSource` API，无需额外库
- HTTP 协议，无需协议升级
- 自动重连（虽然本场景中不需要）

9.2 analyze 处理器流程

```
analyze() 处理器
|
|─ 1. 锁定 AppState
|   |─ 中断已有分析 (api_interrupt.take() → flag.store(true))
|   |─ 验证地图和角色
|   |─ 构建 RoundSpec
|   |─ 调用 prompt::full_request() 组装完整请求
|   |─ 构建消息列表 (system prompt + 历史局 + 本局请求)
|   |─ 清空分析状态 (retry_count=0, plans=[], ...)
|   └─ 返回 (config, messages)
|
|─ 2. 启动 API 线程
|   |─ chat_stream_threaded(config, messages) → (rx, interrupt)
|   └─ AppState.api_interrupt = Some(interrupt)
|
|─ 3. 创建 async_stream::stream! 流
|   |
|   └─ loop {
|       |─ rx.try_recv()
|       |
|       |─ ok(Done(Ok(outcome)))
|       |   |─ interrupted → yield error 事件
|       |   |─ 解析校验 parse_and_validate()
|       |   |   |─ ok(plans) → 计算 usage/cost → yield plans 事件 → return
|       |   |   └─ Err(e)
|       |   └─ retry_count < 3 → 追加错误消息 → 启动新 API 线程 → yield retry
|       |
|       └─ retry_count >= 3 → yield error("不适配") 事件 → return
|
|   └─ ok(Done(Err(e))) → yield error 事件 → return
|
|   └─ ok(Delta(_)) → 忽略（当前未使用增量推送）
|
|
```

```

|         |└ Err(Empty) → tokio::time::sleep(150ms) → continue
|         |
|         |└ Err(Disconnected) → yield error("断开") 事件 → return
|         }
|
|
└ 4. 返回 Sse::new(stream)

```

9.3 重试机制中的 Receiver 替换

重试时需要创建新的 API 调用线程，生成新的 `Receiver`。由于 `Receiver` 存在于 `stream!` 宏的闭包内（而非共享状态），替换通过 `Option<Receiver>` 实现：

```

let mut rx_opt: Option<mpsc::Receiver<api::ApiEvent>> = Some(rx);
loop {
    let recv_result = if let Some(rx) = &rx_opt {
        rx.try_recv()           // 借用 rx_opt
    } else {
        return;                // rx_opt 为 None, 退出
    };
    // 借用在此结束
    match recv_result {
        Ok(ApiEvent::Done(Ok(outcome))) => {
            // ... 校验失败重试 ...
            let (new_rx, new_int) = api::chat_stream_threaded(cfg, msgs);
            rx_opt = Some(new_rx); // 替换 Receiver (旧 borrow 已结束)
            yield sse(...);
            continue;
        }
        // ...
    }
}

```

`try_recv()` 返回的是 `Result`（owned 类型），借用 `&rx_opt` 在 `try_recv()` 返回时即结束。因此可以在 `match` 分支中安全地替换 `rx_opt`。

9.4 打断机制



前端调用 `POST /api/session/interrupt`，服务端将 `AppState.api_interrupt` 中的 `Arc<AtomicBool>` 设为 `true`。API 线程在下次 150ms 轮询中检测到并停止读取，返回 `interrupted=true`。SSE stream 收到 `Done(Ok(outcome))`，检测到 `interrupted` 标志后推送 error 事件并关闭流。

9.5 刷新页面后的中断处理

如果用户在分析过程中刷新浏览器，`EventSource` 连接断开，但 `api_interrupt` 仍留在 `AppState` 中。下次 `analyze` 被调用时，处理器首先执行：

```
if let Some(flag) = s.api_interrupt.take() {
    flag.store(true, Ordering::Relaxed);
}
```

取走旧的 interrupt flag 并设为 `true`，使遗留的 API 线程停止，然后正常启动新分析。

10. 前端实现

10.1 单页应用架构

前端为纯原生 JavaScript 单页应用，无框架依赖。`index.html` 包含全部 12 个步骤界面（`<div class="step">`），默认全部隐藏。`app.js` 通过 `showStep(name)` 函数控制显示/隐藏：

```
function showStep(name) {
    document.querySelectorAll('.step').forEach(e1 => e1.style.display = 'none');
    const e1 = $('step-' + name);
    if (e1) e1.style.display = 'flex';
}
```

10.2 各步骤界面实现

步骤	HTML id	关键交互	API 调用
API 配置	step-config	默认配置询问 / 表单填写	GET /api/config , POST /api/config
历史导入	step-history	历史列表点击导入	GET /api/history , POST /api/history/load
全局需求	step-global-req	文本框输入	POST /api/session/global-req
选地图	step-map-select	3×3 图片网格点击选择	GET /api/resources (缓存)
选角色	step-char-select	6 列网格 + 分页 + 已选栏	(无 API, 本地状态)
当局需求	step-round-req	文本框输入	POST /api/session/round
模型分析	step-analyzing	Spinner + 打断按钮	GET /api/session/analyze (SSE), POST /api/session/interrupt
结果展示	step-result	Canvas + 天赋列表 + 翻页 + 确认	GET /api/positions/{map}
等待结束	step-waiting-end	同结果展示但锁定 + 对局结束按钮	POST /api/session/end-round
下一局	step-ask-next	Token 统计展示 + 是/否按钮	(无额外 API)
保存历史	step-save-history	命名输入 + 保存/不保存	POST /api/history/save
错误退出	step-error-exit	错误消息 + 重试/重输/退出	(无额外 API)

10.3 角色选择分页布局

角色选择界面遵循 `icons/AGENTS.md` 中的布局规范：左 80% 为角色选择区，右 20% 为已选角色展示区。

角色选择区采用 6 列 × 3 行 = 18 格布局，左右各 3×3。51 名角色按 `DICTIONARY.md` 顺序排列，每页 18 个，共 3 页：

```

for (let row = 0; row < 3; row++) {
  for (let col = 0; col < 6; col++) {
    let idx;
    if (col < 3) {
      idx = pageStart + row * 3 + col;          // 左半: 0-8
    } else {
      idx = pageStart + 9 + row * 3 + (col - 3); // 右半: 9-17
    }
    // ...
  }
}

```

10.4 beforeunload 保护

当存在未保存的会话时，浏览器关闭/刷新会弹出确认提示：

```

window.addEventListener('beforeunload', (e) => {
  if (hasUnsavedSession) {
    e.preventDefault();
    e.returnValue = '';
  }
});

```

`hasUnsavedSession` 在收到 plans 事件（分析成功）时设为 `true`，在保存历史或选择不保存时设为 `false`。

11. Canvas 图片合成

11.1 设计对比

桌面版使用 Rust `image` crate 进行像素级 alpha 混合，将角色头像叠加到区域选点底图上，生成单张 `egui::ColorImage` 渲染。WebUI 版改为前端 **HTML5 Canvas** 实现相同效果：

对比项	桌面版	WebUI 版
合成位置	Rust 后端 (<code>compose_map_image</code>)	浏览器前端 (<code>composeMap</code>)
混合方式	像素级 <code>for</code> 循环 alpha 混合	Canvas <code>drawImage</code> (浏览器原生合成)
输出格式	<code>egui::ColorImage</code> → <code>TextureHandle</code>	Canvas 元素
坐标缩放	原始尺寸计算	<code>point.x * scale</code> 计算
切换方案	重新合成整张图片	重新 <code>drawImage</code> (更快)

11.2 composeMap 函数实现

```

async function composeMap(canvas, mapStem, plan, positions) {
  // 1. 加载区域选点底图
  const mapImg = await loadImage(`/icons/area-selection/${mapStem}.png`);

```

```

// 2. 计算缩放比例 (适配展示区域)
const maxW = 700, maxH = 550;
const scale = Math.min(maxW / positions.map_width, maxH / positions.map_height);
const displayW = Math.round(positions.map_width * scale);
const displayH = Math.round(positions.map_height * scale);

// 3. 设置 Canvas 尺寸并绘制底图
canvas.width = displayW;
canvas.height = displayH;
const ctx = canvas.getContext('2d');
ctx.clearRect(0, 0, displayW, displayH);
ctx.drawImage(mapImg, 0, 0, displayW, displayH);

// 4. 计算角色图标尺寸 (随地图缩放)
const iconSize = positions.icon_size * scale;

// 5. 按方案叠加角色头像
for (const sel of plan.selections) {
  const stem = charZhToStem[sel.character]; // 中文名 → 英文名
  const point = positions.points.find(p => p.num === sel.point);
  const iconImg = await loadImage(`/icons/characters/${stem}.png`);

  const cx = point.x * scale; // 选点中心 X (缩放后)
  const cy = point.y * scale; // 选点中心 Y (缩放后)
  const half = iconSize / 2;
  ctx.drawImage(iconImg, cx - half, cy - half, iconSize, iconSize);
}
}

```

11.3 坐标映射原理

`positions.rs` 提供的坐标基于**原始地图尺寸**。Canvas 中的坐标需要按缩放比例换算：

原始坐标系	Canvas 坐标系
$(point.x, point.y)$	$\rightarrow (point.x * scale, point.y * scale)$
图标尺寸	Canvas 图标尺寸
$icon_size$ (原始)	$\rightarrow icon_size * scale$ (缩放后)
图标放置	中心对齐
左上角 = $(cx - half, cy - half)$	

其中 `scale = min(maxW / map_width, maxH / map_height)`，保证地图在最大展示区域内等比例缩放，图标按相同比例缩放。

11.4 天赋图标展示

天赋图标通过 `talentStemMap` (中文名 → 英文 stem 映射) 构建 URL：

```
const talentStemMap = {
  "回光返照": "borrowed-time",
  "飞轮效应": "flywheel-effect",
  "膝跳反射": "knee-jerk-reflex",
  "化险为夷": "tide-turner",
};
```

每个角色的两个天赋图标竖直排列，置于角色头像右侧：

```
<div class="talent-item">
   <!-- 角色头像 -->
  <div class="talent-icons">
     <!-- 天赋1 -->
     <!-- 天赋2 -->
  </div>
  <div class="talent-info">
    <span>佣兵</span>
    <span>回光返照、化险为夷</span>
    <span>5号选点</span>
  </div>
</div>
```

12. 错误处理

12.1 错误类型与处理方式

错误类型	触发条件	SSE 事件	前端处理
API 网络错误	HTTP 请求失败、超时等	<code>{type:"error", message:"..."}</code>	<code>showErrorExit()</code> ：显示重试 / 重新输入 / 退出按钮
模型输出格式错误	parser 校验失败	<code>{type:"retry", count:N}</code>	更新重试提示，继续等待 SSE
重试超限	重试 3 次仍失败	<code>{type:"error", message:"您的模型与该项目不适配"}</code>	<code>showErrorExit()</code> ：不显示重试按钮
用户打断	interrupt flag 被设置	<code>{type:"error", message:"分析已打断"}</code>	<code>showErrorExit()</code> ：显示重试按钮
API 线程断开	mpsc channel Disconnected	<code>{type:"error", message:"API 线程异常断开"}</code>	<code>showErrorExit()</code>
SSE 连接断开	浏览器网络问题等	(无事件, 触发 <code>onerror</code>)	<code>showErrorExit("与服务器的连接断开")</code>
资源加载失败	启动时 <code>ResourceDb::load</code> 失败	(服务端 <code>eprintln</code> 输出)	API 返回空列表
前置条件不满足	未选地图/角色不足	HTTP JSON <code>{error:"..."}</code>	<code>alert()</code> 显示错误

12.2 错误退出界面按钮逻辑

```
function showErrorExit(message) {
  const isModelError = message.includes('不适配');
  const isInterrupted = message.includes('打断');

  // API 错误 (非模型不适配)：显示重试 + 重新输入
  if (isInterrupted || (!isModelError && ...)) {
    // 重试按钮 → startAnalysis()
    // 重新输入按钮 → showStep('map-select')
  }

  // 所有情况：显示保存历史 (如果有已完成局) + 退出
  if (sessionRoundsCount > 0) {
    // 保存历史按钮 → showSaveHistory()
  }
  // 退出按钮 → showSaveHistory() 或 showExitMessage()
}
```

12.3 模型输出校验重试

模型输出校验失败时，SSE stream 中执行重试，对前端透明：

SSE 事件流:

retry {type:"retry", count:1}	← 前端显示"正在重试 (第 1 次)"
retry {type:"retry", count:2}	← 前端显示"正在重试 (第 2 次)"
plans {type:"plans", plans:[...]}	← 前端渲染结果, 关闭连接

前端只需处理 `retry` 事件 (更新提示文字) 和 `plans / error` 事件 (切换界面), 不需要实现重试逻辑本身。

12.4 资源加载失败的降级

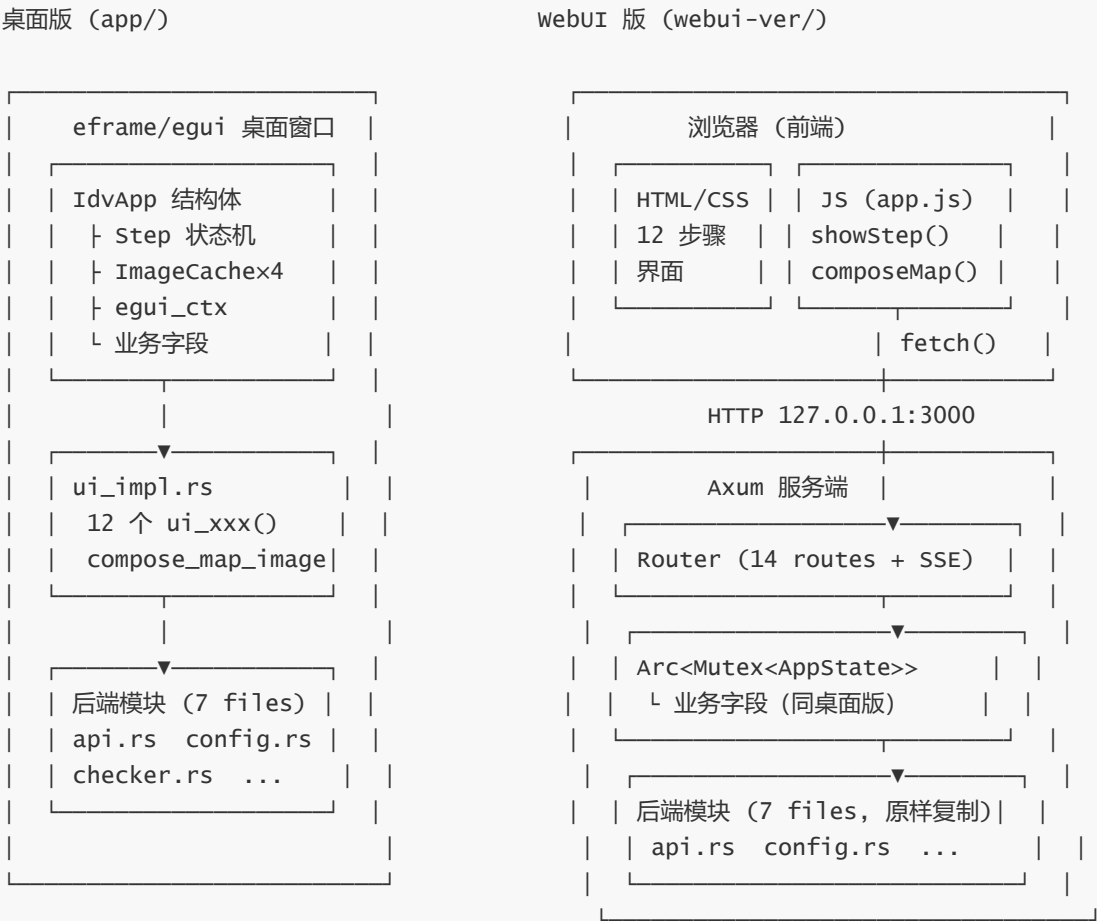
服务端启动时如果 `ResourceDb::load()` 失败 (如资源目录不存在), 会输出错误日志并使用空的 `Vec::new()`:

```
let db = checker::ResourceDb::load(&resources_dir).unwrap_or_else(|e| {
    eprintln!("[IDV-ASAS] 加载资源库失败: {e}");
    checker::ResourceDb { maps: Vec::new(), survivors: Vec::new() }
});
```

此时 `GET /api/resources` 返回空列表, 前端界面会显示空白网格。系统提示词 (`system_prompt`) 也会为空, 后续模型分析会失败。

13. 与桌面版对比

13.1 架构对比



13.2 替代关系映射

桌面版组件	WebUI 版替代	说明
<code>eframe::App</code> trait (<code>logic()</code> + <code>ui()</code>)	Axum 路由 + SSE stream	每帧轮询 → HTTP 请求-响应 + SSE
<code>IdvApp</code> 结构体	<code>AppState</code> (Arc<Mutex>)	移除 <code>egui</code> 字段, 保留业务字段
<code>Step</code> 枚举 + <code>match self.step</code>	<code>showStep(name)</code> (JS)	Rust 状态机 → JS 显示/隐藏 div
<code>ui_impl.rs</code> (12 个 <code>ui_xxx</code>)	<code>index.html</code> + <code>app.js</code>	<code>egui</code> 渲染 → HTML/CSS/JS
<code>ImageCache</code> (TextureHandle)	浏览器 <code></code> + Canvas	GPU 纹理 → 浏览器图片缓存
<code>compose_map_image</code> (Rust 像素混合)	<code>composeMap</code> (Canvas <code>drawImage</code>)	像素级 alpha 混合 → Canvas 原生合成
<code>ctx.request_repaint()</code>	SSE 事件推送	重绘请求 → 事件通知
<code>poll_api()</code> (每帧 <code>try_recv</code>)	<code>stream!</code> 宏循环 (async)	同步轮询 → 异步轮询
<code>ViewportCommand::Close</code>	<code>beforeunload</code> 事件	窗口关闭拦截 → 浏览器离开拦截
内嵌 Deng.ttf	Google Fonts / 系统字体	编译时内嵌 → CSS @import
1280×720 固定窗口	自适应浏览器窗口	固定尺寸 → 响应式
X11 窗口后端	无 (仅需浏览器)	移除图形环境依赖

13.3 后端模块复用验证

```
api.rs:      identical (196 行)
config.rs:   identical (76 行)
checker.rs:  identical (99 行)
positions.rs: identical (138 行)
prompt.rs:   identical (88 行)
history.rs:  identical (92 行)
parser.rs:   identical (211 行)
_____
合计:      7 个文件, 900 行, 零修改
```